Rhizome Testing Day

A manual QA strategy for the rhizome window manager

River + StumpWM-parity stack — Sunday edition

1 Goal and method

Rhizome now has broad feature coverage; this plan puts it through its paces as a **daily driver** under real usage, not just unit conditions. The strategy has three layers:

1. **Regression sweep** — run the automated suite first, so manual testing starts from a known-good base.
2. **Feature circuits** — scripted walks through each feature area with concrete steps and expected results (the bulk of this document).
3. **Soak testing** — use rhizome as your only window manager for the day: browse, edit, chat, take screenshots, switch workspaces under load, and note anything that breaks, flickers, or feels wrong.

Mark each item with a result. Anything that fails: note the section number, what you did, what happened — the capture guide in the appendix turns that into a fixable report.

1.1 Preconditions

☐ Install missing helpers: `sudo pacman -S swaylock swayidle` (lock screen — MOD+Shift+L is a no-op without them). ☐ Fresh login: pick the rhizome (river) session at the SDDM greeter. Watch the first 10 seconds: one bar, one wallpaper process, both autostart apps (variety tray-less in tray, flameshot daemon). ☐ Automated suite green (next section).

1.2 The automated suite (15 min)

Run each from the repo root; every line must end in `0 failed`:

```
for t in workspace-restore workspace-switch-away window-rules \
         hooks-and-eval ipc prefix prompt-and-message floating \
         frames parity quickwins interactivity medium3 bigger1 \
         session-icons bar-guard expose; do
  echo "== $t"; sbcl --non-interactive --load tests/$t.lisp
done
```

Suite	What it covers	Result
frames (57)	Tree ops, dynamic tiling, dumps, refits	☐
expose (22)	Grid, pick paths, exact restore	☐
scroll (27)	Ribbon geometry, clamping, focus recovery	☐
groups (11)	Group types, floating adoption, cycling	☐
dbus (14)	Wire protocol + live broker integration	☐
tray (13)	Watcher service, item registry, click routing	☐
bigger1 (27)	Minor modes, WSE, fuzzy	☐
session-icons (25)	Lock/logout commands, bar icons	☐
the rest (152)	Everything else, per README	☐

Total: 414 checks. If any fail, stop — manual testing is pointless on a broken base.

2 Feature circuits

2.1 B. Dynamic tiling (the new core)

The behavior under the most scrutiny today: every new window should get its own frame.

	Action	Expected
☐	On an empty workspace press MOD+Return twice	First terminal fills the screen; the second splits it side-by-side
☐	MOD+Return again (3rd, 4th, 5th window)	Each arrival splits the frame you are in along its longer axis; the pointer follows the newest window
☐	C-t R (remove split) after several arrivals	Frames merge pairwise into their sibling; windows stack one-per-frame again
☐	Close windows one by one with several open	The empty frame is reclaimed: neighbors grow automatically — no manual C-t R needed
☐	C-t Q (only) with several frames	Whole workspace collapses to one frame; all windows in it, cycle with C-t n
☐	C-t B (balance) after manual resizing	Every split returns to 50/50
☐	Open 8+ windows	Splits stop below twice the minimum frame size (no sliver frames); still stable
☐	C-t n / C-t p inside a multi-window frame	Pulls the next/previous hidden window into the current frame

2.2 C. Frame tree surgery

	Action	Expected
☐	C-t s, C-t S	Manual side-by-side and stacked splits; focus moves to the new empty frame
☐	C-t arrows	Focus moves to the nearest frame in that direction, preferring edge-sharing neighbors
☐	C-t +/-	Dividing line nudges ±30px; iresize (C-t r) then arrows works continuously, Enter commits, Escape cancels
☐	C-t M + arrows	Moves the focused window into that frame
☐	C-t e + arrows	Swaps the current frame's window with the target frame's (focus follows the arrival)
☐	Switch layout: `rhizome-cmd switch-layout master-stack`	Tree rebuilds as master/stack; windows distributed one per frame
☐	`rhizome-cmd dump-frame proj` then open/close windows, then `restore-frame proj`	Layout returns; windows placed by number; missing windows skipped without crash

2.3 D. Expose (C-t Return)

	Action	Expected
☐	With 3+ windows on the workspace: C-t Return	All windows (including hidden-in-frame ones) float into an aspect-fitted grid; a hint message appears

☐	Press 1–9 / 0	That grid slot is picked; layout restores exactly and that window focuses
☐	Arrows, then Return	Focus walks the grid; Return picks the focused window
☐	Click a window in the grid	Pick completes with that window
☐	Escape	Layout and focus restore to pre-expose state exactly — float positions, frame order, focus
☐	Wait 60s mid-expose	Timeout restores safely on its own
☐	One window only: C-t Return	“nothing to expose” message, no state change

2.4 D2. The scroll ribbon (C-t v, niri-style)

	Action	Expected
☐	C-t v with 2-3 windows open	Windows become full-height columns in their tiling order; the bar’s layout module reads “scroll”; a message reports the column count
☐	C-t . / C-t , (period/comma)	Focus moves one column over; the viewport scrolls the minimum distance to keep the focused column fully visible
☐	MOD+. / MOD+,	Viewport scrolls alone in half-screen steps, clamped at both ribbon ends; focus unchanged; the scroll-x message appears
☐	Open a new window while scrolled	It enters as a new column immediately right of the focused one and gets focus
☐	Close the focused window while scrolled	Its column is pruned; focus falls to the vacated slot’s successor, not the first column
☐	Float a column window (C-t f)	It leaves the ribbon (column count drops); the float renders and drags normally
☐	C-t v again	Back to the dynamic tiler; windows re-split into frames as if the ribbon never happened
☐	C-t Return (expose) while scrolled	The overview works over columns; restore returns to the ribbon exactly

2.5 D2b. The tray (StatusNotifier icons)

	Action	Expected
☐	Bar with Variety running	A tray icon appears in the right segment next to the clock; the bar log shows the item registered
☐	Left-click a tray icon	The item’s Activate runs (Variety opens its preferences, flameshot its overlay)
☐	Scroll over a tray icon	Scroll(delta, vertical) reaches the item without errors in the log

☐	Close an app with a tray item	Its icon disappears within a second (NameOwnerChanged fires and the item is pruned)
☐	`busctl --user status org.kde.StatusNotifierWatcher`	PID points at the bar's sbcl process
☐	`rhizome-cmd toggle-bar` twice	The watcher name follows the bar (released on stop, re-acquired on start); icons return

2.6 D3. Group types (C-t l)

	Action	Expected
☐	C-t l with tiled windows	The ribbon appears (tile → scroll); message names the type
☐	C-t l again	Everything floats in a cascade (scroll → floating); message names the type
☐	Open a new window while floating	It arrives floated, cascaded below the existing floats — not tiled
☐	C-t l a third time	Back to dynamic tiling (floating → tile); windows re-split into frames
☐	`rhizome-cmd set-group-type floating` then back with `tile`	Direct type sets behave like the cycle legs

2.7 E. Floats and pointer operations

	Action	Expected
☐	C-t f on a tiled window	Floats in place (no jump); "floated" message
☐	MOD+Left-drag a float	Moves; alpha-blended border follows
☐	MOD+Left-drag a **tiled** window	StumpWM-style: it floats first, then moves
☐	MOD+Right-drag a float	Resizes from the bottom-right; respects min/max size hints
☐	Drag a float mostly off-screen, release	Clamped back so 30px stays reachable
☐	Titlebar-drag a CSD app (GIMP toolbox, if installed)	Moves without floating it again
☐	C-t f again to unfloat	Returns to tiling in the current frame
☐	Drag a float across a workspace switch (MOD+1..9 while dragging is fine to skip)	No crash; float lands on its workspace

2.8 F. Workspaces, rules, marks

	Action	Expected
☐	MOD+1..9 / C-t 1..9	Direct switch; bar highlights; hidden windows restore
☐	MOD+Shift+1..9	Moves the focused window and follows it
☐	C-t N / C-t P	Next/previous workspace
☐	`rhizome-cmd renumber` after closing windows	Numbers compact 1..N; C-t ' selects by number
☐	C-t # then `rhizome-cmd move-marked 3`	Marked window (star) moves; marks clear

☐	Set a rule: `(configure '(:window-rules ((:match "foot" :workspace 4))))`, then launch a terminal	It opens on workspace 4; check via C-t w (windowlist shows workspace numbers)
☐	Same with a `:wse` rule: `(:wse (:and (:app-id "foot") (:float nil)) :float t)`	Matching windows float automatically

2.9 G. Prompts, launcher, menus (fuzzy)

	Action	Expected
☐	MOD+D, type “ff”	“Firefox”-style fuzzy hits rank after exact substring matches; the just-installed Variety appears
☐	MOD+W (windowlist)	Numbers shown as `N: [ws] title`; marked windows show a star; Enter jumps to its workspace and focuses
☐	MOD+X (command palette), type “bal”	Fuzzy filters; Enter runs balance; the message window confirms
☐	C-t ’ then a number	Selects that window by number
☐	Tab/Up/Down in any prompt	Selection cycles; Escape cancels cleanly every time
☐	Leave a prompt open 30s	Auto-closes; keyboard never trapped

2.10 H. Bar

	Action	Expected
☐	Click workspace numbers in the bar	Switches workspaces via IPC
☐	Click the flameshot icon	Opens flameshot gui
☐	Click the variety icon	Launches/raises Variety preferences
☐	Scroll on the volume module	Volume steps 5%
☐	C-t b / MOD+B	Bar hides and returns; frame trees refit — windows never tiled under or floating over the bar
☐	`(configure '(:bar (:height 30)))`	Bar restarts taller; windows re-tile under it immediately

2.11 I. Session management

	Action	Expected
☐	MOD+Shift+L	Locks (needs swaylock installed); password unlocks
☐	MOD+Shift+E once	“press again to log out” message
☐	MOD+Shift+E twice within 3s	Session ends; SDDM greeter returns; log back in cleanly
☐	`rhizome-cmd suspend`	Machine suspends; wake restores session with input working
☐	MOD+Shift+Q / R	Confirm-style behavior consistent; power actions prompt polkit as configured

☐	C-t q twice / once	Quit needs the second press; a single press just warns

2.12 J. Prefix chains, help, modes

	Action	Expected
☐	Press C-t and pause	Which-key list shows every binding with docs (multi-line message window)
☐	C-t h then descend b/c/f/v/k/w	Each shows its output; k then a key describes it without running it
☐	C-t C (command mode), run 3 commands, Escape	Stays armed between commands; which-key visible; Escape exits
☐	`(enable-minor-mode :test)` with a bound key	Mode binding shadows the prefix binding while enabled; disable restores
☐	C-t m (lastmsg), C-t a (time)	Both show in the message window; lastmsg re-shows the previous text
☐	Unbound key in a chain	"unbound key" message, clean exit
☐	`(set-prefix-key "C-z")`, try C-z then C-t	C-z now opens chains; C-t is inert; set it back with `"(rhizome:set-prefix-key \"C-t\")"`
☐	`(set-user-mod :alt)` and back to `:super`	MOD bindings re-grab in both directions; MOD+drag follows

2.13 K. IPC and REPL robustness

	Action	Expected
☐	`rhizome-cmd current-workspace`, `wse (:float t)`, `renumber`	All reply in the message window without breaking the WM
☐	`rhizome-cmd '(+ 1 2)'`	Evaluates; reply shows in the message window
☐	M-x slime-connect (or `rhizome-cmd`) while windows are moving	No request from a foreign thread errors in the log
☐	Flood: hold MOD+J/K for 10s, then C-t chains immediately	No input loss, no stuck modes
☐	`rhizome-cmd lock` twice in a row	Second lock doesn't spawn duplicates (swaylock may refuse politely)

3 Soak testing (the afternoon)

Use rhizome as your only session for 3–4 hours. Keep this list open and note anything that matches:

Watch for	Notes
☐ Flicker	Bar redraws, message window flashes, window borders re-drawn repeatedly — check the WM log for manage/render churn (a past bug class)
☐ Focus fights	Auto-focus stealing, clicks not moving focus, expose leaving stale focus
☐ Slow chains	C-t taking > 100ms, launcher scrolling hitching
☐ Workspace drift	Windows landing on wrong workspaces via rules, or losing their frame order after switches

☐ Log growth	Tail `~/.cache/rhizome/rhizome.log` and `bar.log` — idle should add nothing; bursts during drag/resize are expected but should stop on release
☐ Memory	`ps -o rss -p $(pgrep -f rhizome:run)` at start vs end of the afternoon; >2x growth is worth investigating

Suggested load: 8–12 windows across 4+ workspaces, a browser with video, an editor, two terminals, one GIMP-style float if available. Exercise expose, splits, and workspace switches while the video plays.

4 Advanced circuits (if the morning goes well)

- **Stability — protocol stream (the freeze fix):** open and close the launcher, show messages (C-t m), open prompts (MOD+W) repeatedly — 10+ popups. Then check the log: `grep -c "Malformed" ~/.cache/rhizome/rhizome.log` — reads 0 after the v5 compositor cap (any non-zero number means the desync is back; stop and report).
- **Stability — the snapshot storm (the freeze fix):** with the bar running and windows open, run `ls -l --time-style=+%s ~/.cache/rhizome/snapshot.sexp` twice, 10 seconds apart — the mtime must be IDENTICAL while nothing changes (a bumping mtime at idle = the bar feedback storm). Also: `tail -f ~/.cache/rhizome/rhizome.log` should be quiet at idle — a stream of MANAGE-START lines at idle is the storm.
- **Stability — scroll gauntlet:** toggle C-t v, wait 10s (the message expiry), toggle back, wait 10s. The laptop must stay responsive and the log quiet; this exact sequence froze the machine before commit b1e7996.
- **Session — portal dialogs (the chooser fix):** in Steam, open Settings → Storage → add a library folder — the omanta file picker must appear. This exercises D-Bus-activated portal backends, which died silently before the session imported its environment (river/init now mirrors Omarchy's dbus-update-activation-environment and /.config/xdg-desktop-portal/river-portals.conf pins the backends). Any app's file chooser counts: Firefox uploads, Chromium saves — if a picker never appears, the activation environment went stale again.
- **Session — systemd services (the anchor):** after login, `systemctl --user is-active graphical-session.target` must read `active`, and the session set must be running (`systemctl --user list-units | grep -cE 'fcitx5|sleep-lock|crash-watch|owed|voxtype'` — at least 5). Before the rhizome-session.target anchor, none of these omarchy services ran in a river session at all. After a MOD+Shift+E logout and re-login, the services must come up exactly once — no duplicates.
- **Session — polkit prompt (the agent):** run `pkexec true` — the lxqt-policykit dialog must appear and, after entering the password, exit 0. Without a registered agent polkit falls back to a textual prompt that cannot render under Wayland, and pkexec fails with "Error creating textual authentication agent". This is also what the MOD+Shift+Q / R power rows exercise.
- **Hot-reload resilience:** `(rhizome:configure '(:bar (:height 26)))` from `rhizome-cmd`, then `ql:quickload` a changed file via the eval prompt (C-t :). The WM should survive; note any stuck mode or lost focus. Recovery lever: `C-t q q` then relogin.
- **Multi-bar safety:** confirm exactly one bar process at all times: `pgrep -fc "rhizome:run-bar"` must read `1` (the fork-bomb guard).
- **Timeout paths:** enter iresize, expose, command mode and just wait 60s in each — all three must restore their state cleanly on expiry.
- **Float + expose interplay:** a pre-existing float plus dynamic tiles, expose, pick, repeat — float geometry must survive every round trip.
- **Maximize round-trip:** on a tiled window run `rhizome-cmd maximize` — it must float covering the output minus the bar strip; maximize again must return it to the tiling. Repeat with a pre-existing float: maximize must save and restore its exact geometry. A client-initiated maximize (terminal set_maximized, browser maximize button) must land in the same state.

- **Theming — fonts and colors:** change `(rhizome:configure '(:theme (:font-size 16)))` live — borders and the in-WM surfaces (message window C-t m, prompt MOD+W, launcher MOD+D) must re-render with the larger font immediately; `toggle-bar` twice must restart the bar with the new font. Set `:theme :font-family "nonexistentxyz"` — fontconfig must fall back to a system font, never a crash or blank bar. Override `:launcher :background` and `:theme :focused-border` and confirm both surfaces picked them up on the next redraw.
- **Workspace indicator:** with windows on workspaces 1 and 2 only, the bar must show 1 and 2 in the foreground color, 3-9 muted, and the active workspace underlined in accent with visible gaps between numbers. Switch workspaces (MOD+N and MOD+J/K chains) — the underline must follow within one bar poll (0.15s), the occupancy colors must track window opens/closes/moves, and minimizing the only window on a workspace must mute it.
- **Bar desync soak (the v5 cap):** after a bar restart, run `grep -c Malformed ~/.cache/rhizome/bar.log` after 5 minutes of normal use — must read 0. The bar process binds wl_compositor at v5 (wm-protocol.lisp has the same cap) because wayflan cannot demarshal wl_surface v6 events; a single v6 event desyncs the bar's stream and kills it.
- **Pointer warp on switch:** MOD+2 — the pointer must jump to the output's center within the same frame as the workspace change. Disable it live (`(rhizome:configure '(:pointer (:warp-on-switch nil)))`) and confirm MOD+3 leaves the pointer where it was. Re-enable after.
- **configure_bounds (new windows):** open an app that tends to come up huge (an image viewer with a large image, some terminal profiles) — its initial size must stay within the screen even before rhizome proposes tiled dimensions.
- **Recording indicator:** if `:recording` is in a bar segment, start a screencast (portal-based screen recorder) — a red REC must appear within one bar poll and vanish when it stops.
- **Remembered placements (v4 identifiers):** open an app, move it to workspace 5, float it, close it — then reopen it: it must land back on workspace 5, floating, without any rule. Log out and back in with it open: same result. `cat ~/.local/share/rhizome/window-state.sexp` after closing shows the entry.
- **Presentation mode:** `(rhizome:configure '(:output (:presentation-mode :async)))` — the log must show the mode sent on the next render; a fast-scrolling page should now tear. Revert to `:vsync` and confirm the resend in the log.
- **xcursor theme:** set `(:pointer (:xcursor-theme "Adwaita" :xcursor-size 32))` before login — compositor-rendered cursors (resize edges, window borders) must follow; client-drawn cursors may not (per the protocol's own note).
- **WM hot-swap (detach):** open a few windows, note their workspaces, then `rhizome-cmd detach`. The bar must vanish, the WM process must exit ("detach confirmed" in `~/.cache/rhizome/rhizome.log`), and river must stay up with the windows frozen in place. Then take over again from a terminal: `sbcl --eval '(ql:quickload :rhizome)' --eval '(rhizome:run)' &` — the bar must come back and identified windows must land on their remembered workspaces. This is river's headline feature; if it works, everything above it works.
- **Workspace types:** `rhizome-cmd set-workspace-type floating 4` — every window on workspace 4 must float immediately; opening new windows there must float them too. `set-workspace-type tile 4` must return them all to the grid. Per-workspace layouts: MOD+Space to :even on workspace 1, switch to workspace 2 — it must still show its own layout; switch back — :even must be waiting. A restart of the WM must remember both (config-declared types via (:workspaces (:types ...)) and layouts per-workspace in-session).
- **Gaps:** tiled windows must show breathing room ((:gaps :inner 6), default): twice the gap between adjacent windows, once at screen edges, none around floats. Set (:gaps (:inner 0)) and confirm the classic flush tiling returns on the next manage pass.
- **Translucency:** the bar, launcher (MOD+D), message window (C-t m) and prompts (MOD+W) must render translucent ((:bar :opacity 0.9) and (:launcher :opacity 0.9), default) with correctly-weighted text — no washed-out colors (the buffers are premultiplied ARGB). Set (:bar (:opacity 1.0)) live and confirm surfaces go opaque without a restart (next redraw).

- **Launcher icons:** every entry must show its .desktop icon left of the text, vertically centered, with text aligned across iconless entries. (:launcher :icon-size 20) scales them.
- **Animations:** MOD+2 — the new workspace's windows must glide in from the screen edge (direction follows the switch), landing exactly in place within 200ms; no window may freeze mid-air (a missed frame snaps to the target). C-t Return (expose) — windows glide into the grid; pick one — it glides back. `rhizome-cmd maximize` glides to full-screen and back. Set `(:animations (:enabled nil))` live and confirm everything becomes instant (the old behavior); re-enable after.
- **Workspace-switch soak (the WM-death regression):** the exact scenario that terminated the WM twice: MOD+2, open a window there (Vivaldi), MOD+1 back — the WM must survive. Then rapid round-trips (2, 1, 3, 1 in fast succession, interrupting animations mid-flight) ×10; `grep -c 'invalid modification' ~/.cache/rhizome/rhizome.log` must not grow, and no fractional type errors (`is not of type (SIGNED-BYTE)` may appear. The failures were buffer-timing dependent: SPEED IS THE TEST.
- **MOD+Q closes cleanly:** close a window via MOD+Q and via `rhizome-cmd close` — the window must vanish within one manage cycle and the WM survive both paths.
- **Pointer window menus:** MOD+Middle-click on several windows (one tiled, one floating) — a context menu opens AT the pointer with Minimize, Float/Tile, Maximize, Fullscreen, Close, and Move-to-workspace entries. Hover must highlight a row (accent edge); click must run the action (verify Minimize hides and Restore returns it, Move-to-workspace relocates and the WM survives); Escape must dismiss; moving the pointer off the menu must dismiss it. Arrows + Enter select via keyboard. The close path routes through the manage-sequence queue — the WM must survive every action here.
- **C-t F11 fullscreens on the FIRST press:** the fullscreen request flush was once inverted (first press sent a no-op exit-fullscreen; only the second worked). Press C-t F11 once — fullscreen immediately; press again — back. Both directions on the first press, no double-press workaround.
- **Built-in screenshot (bar included):** `MOD+Print` / `rhizome-cmd screenshot` captures the full composited output INCLUDING the bar (grim copies the output frame — layer surfaces are part of it). For a capture of an overlay state: `sleep 8 && rhizome-cmd screenshot` in a terminal, then open the target state (workspace
  1. MOD+D launcher with a filter typed in) before the timer fires.

  Flameshot on river is best-effort only: its editor floats via window rule; prefer the built-in path for anything important.
- **Pseudo-shadows on floats (river_decoration_v1):** MOD+drag a tiled window to float it — a soft shadow band must appear around it immediately (12px default, accent-tinted when focused: focus another window and the shadow's color shifts). Resize the float (MOD+Right-drag) — the shadow follows with no flicker. Un-float (menu → Tile, or `C-t f`) — the shadow must vanish. Close a floated window — the WM survives; `grep decoration ~/.cache/rhizome/rhizome.log` stays clean of errors. Tweak via `(:decorations (:radius 14 :alpha 0.5 :focused-alpha 0.7))`; `(:enabled nil)` makes floats flat again.
- **Tray right-click menus (dbusmenu):** right-click a tray icon on the bar (flameshot, variety) — the app's own context menu opens AT the icon (the pointer menu: hover highlights, click selects, Escape or pointer-off dismisses). Selecting an entry performs the app's action (e.g. flameshot's Capture / Open Launcher / Quit). Items with submenus appear as indented entries (a flat menu can't nest); separators and disabled entries are skipped.
- **Animation pump is quiet (performance):** switch workspaces a few times with apps open, run an expose — the glides must be smooth AND the WM log must stay nearly silent during them (the per-event [debug] lines are gated behind (:debug (:events t)); hundreds of lines per second of glide means the gate broke). During a sustained animation, CPU use stays modest (check with htop — the WM process should be well under a core).

- **WM hot-swap (the live upgrade):** `rhizome-cmd detach` — the WM serializes the session (placements, frame trees, per-output workspaces, focus), completes river's stop/finished handshake, and exits; the river-init supervision loop relaunches a fresh process 15s later. Expect: 10-20s of NO keybindings during the boot (river keeps rendering; the mouse works), then the desktop back EXACTLY as it was — same workspace showing, same windows in the same workspaces and frames, focus restored, bar back. The takeover file (/.local/share/rhizome/takeover.sexp) is consumed by the restore. The same loop recovers a crashed WM — a WM death is a blip, not a hard reboot.

5 Appendix A: capture guide (how to report a failure)

- Section number + exact steps + what happened vs expected.
- `tail -50 ~/.cache/rhizome/rhizome.log` (WM) and `~/.cache/rhizome/bar.log`.
- For input/mode bugs: which modes were active (C-t h b lists minor modes), whether a timeout or manual exit was involved.
- One-liner repro if you can: the `rhizome-cmd` or key sequence.

6 Appendix B: quick reference

Key	Action	Key	Action
MOD+Return	Terminal	C-t s / S	Split / stacked split
MOD+J / K	Cycle frame window	C-t R	Remove split
MOD+W / G / X	Windowlist / picker / palette	C-t +/-	Resize split
MOD+D	Launcher (fuzzy)	C-t r	Interactive resize
MOD+B	Toggle bar	C-t Q	Only (one frame)
MOD+1..9	Workspace	C-t B	Balance splits
MOD+Shift+1..9	Move window → ws	C-t Return	Expose
MOD+Shift+L	Lock	C-t f	Float/tile
MOD+Shift+E	Logout (×2)	C-t #	Mark window
C-t q	Quit (×2)	C-t m	Last message
C-t h	Help chains	C-t C	Command mode

Text size: 10pt. Print me, tick me, break things.